T4 사용법

강사님 Mission2B + 도구 1개(Tool) · 짧은 명령

T4/Mission2B (경기) · T4/Tool (설정·테스트)

구성 · EEPROM

- 폴더

폴더	여는 파일	용도
T4/Mission2B	`Mission2B.ino`	경기 (강사님 코드, SW1로 시작)
T4/Tool	`Tool.ino`	자세 저장, PSD·카메라·바퀴·집기 테스트 (강사님 도구 6개 합침)

T3 대비: 1번 관절 오프셋 380 (우리 로봇 정면 = 2047), Tool 에 `ours` 명령 (v27 팔 자세 넣기)

- EEPROM 이란
  - 아두이노 보드 안의 작은 저장 공간 (4KB). **전원을 꺼도, 다른 코드를 올려도 안 지워짐**
  - 강사님 코드는 팔 자세를 코드가 아니라 EEPROM 에 저장 (자세 1개 = 관절 4개 값)
  - 그래서 Tool 로 자세를 저장 → Mission2B 를 올리면 Mission2B 가 그 자세를 읽어 씀
  - 보드를 바꾸면 자세를 다시 저장해야 함
- 업로드 순서

1. Tool 업로드 → 자세 저장, 값 측정 (시리얼 115200, 줄바꿈 켜기 · 블루투스도 됨)
2. 측정값을 `Mission2B.ino` 에 적기 → Mission2B 업로드 → SW1

Tool 명령

명령	동작
`h`	명령 목록
`r` / SW1	팔 현재 값 4개
`t` / SW2	팔 힘 빼기 ↔ 넣기
`p 6`	자세 6번 재생
`s 6`	지금 팔 값을 자세 6번에 저장
`x 6`	자세 6번 지우기
`l`	자세 36개 목록 + 빠진 개수
`o` / `c`	그리퍼 열기 / 닫기 (Devices.h 값)
`g 740`	그리퍼를 740 으로 (값 찾기용)
`psd` / `psd 20`	PSD 5번 / 20번 (평균, FR−FL 포함)
`cam`	블록 sig, x, y, dx, mm, ROI 안/밖
`id`	응답하는 모터 번호

명령	동작 (강사님 함수)
`f 900` `b` `lf` `rt`	전진 / 후진 / 좌 / 우 mm
`rl` / `rr` [보정°]	좌 / 우 90° 회전 (RotateLeft90Degree)
`fa 315` [허용]	FR 앞뒤 (PerformFrontMoveAlign)
`sa 558` [허용]	SL 좌우 (PerformLeftMoveAlign)
`ra 275` [허용]	SR 좌우 (PerformRightMoveAlign)
`ma 311 8`	FL·FR 평균 + 수평 (PerformMoveRotateAlign)
`k 1`~`3`	층 집기: 열기→APP→GRIP→닫기 →OUT→SEARCH
`d 1`~`16`	낙하: A1~8, B1~8 = 9~16
`w`	바퀴 비율 보기
`wf 1 .98 .99 .99`	전진 비율 FL FR BL BR (wb wl wr)
`baud`	모터 57600 → 1M 변경

맞춤 명령(fa sa ra ma) 끝나면 PSD 값 1줄 자동 출력. 1층 = 맨 아래(L1), 3층 = 맨 위(L3)

자세 31개 (EEPROM ID)

ID	이름	언제
4	SEARCH *	운반 자세 (집은 뒤·이동)
8 9 10	L1 APP / GRIP / OUT *	1층(맨 아래) 집기
5 6 7	L2 APP / GRIP / OUT *	2층 집기
28 29 30	L3 APP / GRIP / OUT *	3층(맨 위) 집기
33 32 31	SCAN L1 / L2 / L3	층별 카메라 탐색
27	ORDER	미션지시 순서 읽기
34	START	출발 자세

ID	이름	언제
11~18	A1~A8	Zone A 칸 낙하
19~26	B1~B8	Zone B 칸 낙하 (90° 두 번 돈 뒤 정면)
1 2 3 35 36	—	안 씀

- ***** 표시: 경기 때 1번 관절(밑동)을 정면(2047)으로 고정 → 저장할 때 밑동 각도는 무시됨. `p` 도 똑같이 재생
- APP = 칸 앞 같은 높이, GRIP = 블록 잡는 위치, OUT = 잡고 빼는 위치
- 경기 시작 전 자가점검: SEARCH(4)와 배정표에 쓴 낙하 칸이 없으면 `Missing pose ID n` + 정지
- 탐색 순서: 홀수 열 위→아래, 짝수 열 아래→위 (지그재그)

자세 저장 방법

- 먼저 확인 (중요)

  강사님 코드는 팔 모터 설정이 우리 예전 코드와 다름 (위치 모드 + 오프셋, 범위 제한).

  1. `t` 로 힘 빼고 팔을 손으로 끝까지 움직이며 `r`
  2. 값이 끊기지 않고 범위 안인지: j2 1024~3474 · j3 211~2640 · j4 722~3161
  3. 4095 → 0 처럼 튀거나 범위 밖이면 **자세 저장 전에 알려주기** (바닥 박힘 위험)

- 저장

  1. `t` (힘 빼기) → 손으로 자세 잡기 → `t` (힘 넣기, 그 자리 유지)
  2. 필요하면 `r` 로 확인 → `s 번호`
  3. `p 4` → `p 번호` 로 재생 확인. 블록 집기는 `k 1`, 낙하는 `d 1`
  4. 다 하면 `l` → `missing (mission) 0` 확인

  강사님 방식(poseMon 16진수 → writeEEPROM 표 → 업로드)과 저장 형식이 같아서 Mission2B 가 그대로 읽음

우리 값(v27) 넣기

- 바꾼 것
  - 우리 코드 팔 값 + 강사님 오프셋 { 380, 238, 799, 0 } 으로 변환해 넣음
  - 1번 관절 오프셋 0 → **380**: 우리 로봇 정면(1667)이 강사님 정면 2047 이 되게 (집기·운반 때 2047 고정이라 필요)
- 순서
  1. 로봇 **전원 완전히 껐다 켜기** (우리 코드의 모터 설정이 남아 있으면 팔이 크게 돌 수 있음)
  2. Tool 업로드 → `t` 힘 빼기 → 손으로 travel 자세 → `r`
  3. **1051, 2116, 417, 930** 근처(±50)면 변환 맞음 → `ours`. 많이 다르면 값 보내기
  4. `t` 힘 넣고 `p 4` → `p 27` → `p 33` … 하나씩 확인, 틀린 것만 `s` 로 다시 저장

ID	우리 자세	ID	우리 자세
4, 34	travel	27	order
33 32 31	scan1 2 3	8 9 10	pa1 pg1 pa1
5 6 7	pa2 pg2 pa2	28 29 30	pa3 pg3 pa3
11~18	za1~8	19~26	없음 (강사님 B 는 정면 접근이라 새로)

B 칸이 비어 있으면 배정표에 B 를 쓰는 동안 자가점검에서 멈춤 → B 저장 전엔 배정표를 A 만으로

Mission2B.ino 값 (PSD)

이름	지금	어디서	재는 법
PSD_OBSTACLE_FR	590	장애물 라인 앞	FR
PSD_MISSION_SL	476	미션지시 Zone	SL
PSD_DROPBOXA_SEARCH_SL	558	탐색선 (Zone A 옆)	SL
PSD_DROPBOXA_DROP_SL	632	Zone A 낙하 위치	SL
PSD_STORAGE_FR	315	적재함 칸 앞뒤	FR
PSD_STORAGE_FRONT_AVG / ROT_ADJUST	311 / 8	적재함 2·3열 정면	avg / FR−FL
PSD_STORAGE_CELL2_SL	200	2열 좌우	SL
PSD_DROPBOX_B_FRONT_AVG / ROT_ADJUST	591 / 6	Zone B 정면	avg / FR−FL
PSD_DROPBOXA_FRONT_AVG / ROT_ADJUST	285 / 13	Zone B 복귀 중 A 정면	avg / FR−FL
PSD_RETURN_A_FR	194	B 복귀, 2열 앞	FR
PSD_RETURN_STORAGE_SR	275	B 복귀, 적재함까지	SR

로봇을 맞는 위치에 손으로 두고 `psd` → 값 적기 → 그 값으로 `fa` `sa` `ra` `ma` 를 다른 위치에서 실행해 돌아오는지 확인

Mission2B.ino 값 (거리 · 기타)

이름 (mm)	지금
mm_F_BLINKING_to_OBSTACLE	462
mm_L_OBSTACLE_to_MISSION	304
mm_F_MISSION_to_DROPBOX	858
mm_R_DROPBOX_to_CELL1	110
mm_R_CELL1_to_CELL2 / 2_3 / 3_4	67 / 68 / 67
mm_L_CELL1_to_DROPSIDE	124
mm_F_TO_DROPBOX_B	366
mm_B_DROPBAX_to_FINISH	1320
mm_R / F_DROPBOX_B_to_FINISH	290 / 950

이름	지금	파일
PIXY_SET_POINT_X	158+8	Mission2B.ino
GRIP_ANGLE_CLOSE	955	Devices.h
GRIP_ANGLE_OPEN	100	Devices.h
flMotRateFW …		Mobilebase.h

- 거리: `f 462` 처럼 실제로 가 보고 줄자로 확인
- 카메라 x: 그리퍼와 블록이 일직선일 때 `cam` 의 x → `PIXY_SET_POINT_X`
- 그리퍼: `g` 로 찾기 (예전 측정: 닫기 740 근처, 750 넘으면 서보 소리). Devices.h 는 **Mission2B, Tool 두 폴더 다** 고치기
- 바퀴: `f 1000` → 휘면 `wf …` 로 바꾸고 반복 → `w` 값을 Mobilebase.h 에 적기 (두 폴더 다)

경기 실행 (Mission2B)

- 배정표 (Mission2B.ino, missionAssignments)

  `{ 색, 칸 }` 한 줄 = 블록 1개. 색 1빨 2주 3노 4초 5파 6보. 칸: A = 1~8, B = 8+칸 (예: B5 = `8+5`)

- 스위치 (#define, 파일 맨 위)

이름	기본	뜻
REAL_WCRC_GAME_MODE	true	false 로 해야 아래 두 개가 동작
STOP_BEFORE_DROP	false	낙하 전 멈춤 (자세 확인)
PAUSE_AT_EACH_STEP	false	단계마다 SW1 대기
USE_SHORT_CUT_PASS	false	true: 적재함부터 시작 (경기 땐 false!)
RUN_TOP_ROW_SEARCH	true	false: 3층 안 찾음
RUN_CAM_X_ALIGN	true	카메라 좌우 보정 (최대 3번)
CAM_LAMP_ON	false	순서 읽을 때 램프

- 순서

  업로드 → 자가점검 (실패: 부저 + 빨간 LED + 시리얼 메시지) → START 자세 → **SW1** 시작. 순서 인식 실패 시 삑 3번 + 배정표 순서로 진행

하루 순서

#	할 일	명령
1	Tool 업로드, 모터 8개 확인	`id`
2	팔 값 범위 확인 (5쪽)	`t` `r`
3	그리퍼 값 → Devices.h (두 폴더)	`g`
4	바퀴 직진 → Mobilebase.h (두 폴더)	`f 1000` `lf 300` `wf`
5	자세 31개 저장 + 확인	`s` `p` `k` `d` `l`
6	PSD 값 11개, 거리, 카메라 x	`psd` `cam` `fa` `sa` `ma`
7	Mission2B.ino 에 값·배정표 적고 업로드	SW1
8	STOP_BEFORE_DROP 켜고 1판 → 끄고 반복	

값을 보내 주면 Mission2B 에 넣어 드림 (T4)